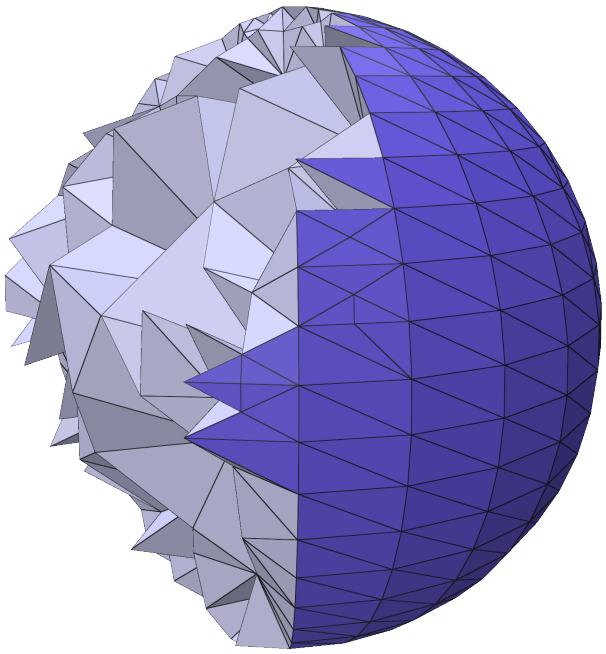
Constrained Delaunay tetrahedrization refinement

Internship report

Max Royer



Supervised by Marco Attene and Marco Livesu

ENS of Lyon
September 2025

Table of contents

1 Introduction

This report summarizes my internship at the *CNR IMATI* in Genova, Italy. It was a pleasure to share those three months with Marco, Marco, and the team (where there are three other Marcos)!

My internship focused on computational geometry, the field in which both Marco Attene and Marco Livesu are working. Recently, Lorenzo Diazzi, Marco A., et al. published an article [1] discussing constrained Delaunay tetrahedralization. So far, they have developed a robust tetrahedralization pipeline using robust geometric predicates developed by Marco Attene. However, they still lacked a refinement process for the tetrahedralized output that other well-known software like TetGen [4] or CGAL [3] provide.

Constrained Delaunay tetrahedralization (CDT) has a wide range of applications. It is commonly used in physics-based simulations such as the finite element method, solutions of partial differential equations, or ray tracing. However, these simulations and applications require high-quality meshes, which raw CDTs often fail to provide. Therefore, a refinement process is essential to improve the overall mesh quality and ensure the robustness and accuracy of the simulations.

The objective of this internship was to design and validate a greedy algorithm for refining Constrained Delaunay Tetrahedralizations (CDTs). We drew major inspiration from the article [2], which already proposed a refinement process but allowed slight deformation of the input surface. In our case, the constraints are strict, meaning we cannot remove an input vertex or change the geometry of the faces. We are only allowed to add vertices, which may lie on the surface of the model only if they do not change the original surface geometry.

The plan of this report is to first present some preliminaries and the state of the art, then to present the refinement process, and finally to compare it with other existing processes.

State of the Art

The state of the art, for us, is composed of two pieces of work:

TetGen [4] is software published in 2015 by Hang Si, which is still a reference regarding constrained Delaunay tetrahedralization. Similar to our project, TetGen has a refinement process at the end of their pipeline where they apply local operations. The issue is that TetGen is not very robust and often crashes or does not converge, thus never finishing. One of the objectives of our algorithm is to be robust.

Tetrahedral meshing in the wild (TetWild) [2] is an article published in 2018 by Hu et al. in which they introduce the Dirichlet energy that we are also using in this project, along with a first set of local operations. In their case, they allowed themselves to slightly deform the geometry of the input, thereby achieving the ability to "correct" some intrinsic malformations of the input (e.g., the model has a very sharp needle). This is something we want to avoid, so one of our other objectives is to be strict on the boundary and not allow any deformation. This may lead to having some elements that cannot be refined.

In the paper, Tetwild implements the edge split, the edge collapse, some face swaps, and the vertex smoothing. More precisely, the face swaps they implemented were 3-2, 4-4, and 5-6 bistellar flips, which are particular and precise flips. In our case, we will only use 2-3 bistellar flips (§3.2.3), and I will introduce the edge swap (§3.2.4) later on, which is a more general swap/flip method. In our algorithm, we are also introducing tetrahedron collapse (§3.2.6). Finally, in our case, all operations can't move any input vertices around.

On the other hand, the *CDT* software published by Lorenzo Diazzi, Marco A., et al. [1] is robust and exact on the boundaries but produces low-quality meshes, whereas *TetGen* and *TetWild* produce high-quality meshes. The goal is to offer a refinement process that is still exact and robust to produce high-quality meshes for *CDT*.

2 Preliminary

2.1 Piecewise Linear Complex

In three dimensions, piecewise linear complexes are extremely flexible representations that allow to model not only 3D surfaces, but also more generic collections of 2D "constraints" that we may want to keep in the final 3D mesh.

Definition 2.1 (Piecewise Linear Complex)**.** A Piecewise Linear Complex (PLC) X is a finite set of vertices, segments and facets of $\mathbb{R}^3$. A facet f is a polygonal region that can have holes and any number of sides, and so is not necessarly convex. Then X must respect those two rules:

- X must be closed by intersection, it means that when intersecting two elements of X the intersection is either empty or is an element of X.
- Every facet of X must be planar.

Here is an example of PLC in figure 1.

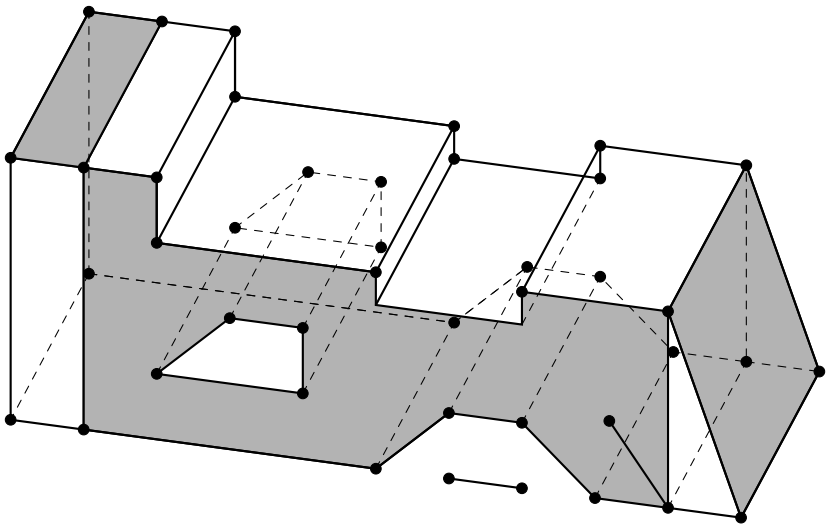


Figure 1: Example of PLC, courtesy of Hang Si

2.2 Delaunay's triangulation

Definition 2.2 (Simplex)**.** A simplex σ of dimension d is the convex hull of $(v_0, v_1, \cdots, v_d)$ points of $\mathbb{R}^n$, such that $(v_1 - v_0, \cdots, v_d - v_0)$ are linearly independant. The v_i's are called the vertices of σ, and we say that σ is spanned by $V := \{v_0, \cdots, v_d\}$. A face of σ is a simplex spanned by a subset of V, we denote the fact that τ is a face of σ by writing $\tau \leq \sigma$.

Remark 2.1. In dimension 1 a simplex is an segment, in dimension 2 it's a triangle and in dimension 3 it's a tetrahedron.

Definition 2.3 (Simplicial complex)**.** Let K be a set of simplices, we say that K is a simplicial complex if:

- If $\sigma \in K$ then for all τ such that $\tau \leq \sigma$ we have $\tau \in K$.
- Let $\sigma, \sigma' \in K$ then $\sigma \cap \sigma'$ is either empty or a face of both.

Definition 2.4. Let K be a simplicial complex, and let $B \subseteq K$:

- The *closure* of B is :
$$\overline{B} := \{\tau \in K \mid \exists \sigma \in B, \tau \leq \sigma\}$$
- The *star* of B is :
$$\mathrm{St}(B) := \{\sigma \in K \mid \exists \tau \in B, \tau \leq \sigma\}$$
- The *link* of B is :
$$\mathrm{Lk}(B) := \overline{\mathrm{St}(B)} \backslash \mathrm{St}(\overline{B})$$
- We also define for simplicity the *vertices* of B that are :
$$\mathrm{Vert}(B) := \{v \in B \mid \dim v = 0\}$$
and the *edges* of B which are :
$$\mathrm{Edges}(B) := \{e \in B \mid \dim e = 1\}$$

Definition 2.5. Some additional notation may be useful in the following, let K be a simplicial complex:

- We define the incident Vertex Tetrahedra's of $v \in K$ where $\dim v = 1$ to be:
$$\mathrm{VT}(v) := \left\{t \in K \mid \dim t = 3 \text{ and } v \in \overline{\{t\}}\right\}$$
- We define the incident Edge Tetrahedra's of $e \in K$ where $\dim e = 2$ to be:
$$\mathrm{ET}(e) := \left\{t \in K \mid \dim t = 3 \text{ and } e \in \overline{\{t\}}\right\}$$

Definition 2.6 (Delaunay triangulation)**.** For $P \subseteq \mathbb{R}^d$, a valid Delaunay triangulation is a simplicial complex $DT(P)$ such that for any $\sigma \in DT(P)$, the interior of the *circum-hypersphere* of σ is empty. The later condition is often denoted as the Delaunay condition.

Remark 2.2. In our case we'll only focus on dimension 3, in this case a Delaunay tetrahedrisation exists and is guaranteed to be unique if and only if the points are non coplanar and if no 5 points are cospherical.

2.3 Constrained Delaunay tetrahedrisation

While a Delaunay triangulation provides a structure over a set of points, a "constrained" Delaunay triangulation provides a structure over a set of points and a set of "constraints". In 2D, constraints are segments connecting pairs of the input points. In 3D, conjstraints can be either segments and/or polygons that, together with the vertices, form a PLC.

Definition 2.7 (Locally Delaunay)**.** A triangle τ is said to be locally Delaunay if one of the two following conditions hold:

- It is on the boundary (*i.e.* it is the face of only one tetrahedron)
- It is internal, and the circumsphere of each of its two incident tetrahedra does not contain the opposite vertex of the other

Definition 2.8 (Constrained Delaunay tetrahedrisation)**.** Let $X = (V, E, F)$ be a PLC where V is the set of vertices, E is the set of segments, and F is the set of polygonal facets. A constrained Delaunay tetrahedrization C of X is a 3D simplicial complex subdividing the convex hull of X such that:

- The vertex set V' of C contains V
- Any triangular face of C either is part of a facet in F or is locally Delaunay

2.4 Quality Measure

To measure the quality of our refinement we need a quality measure. A good quality measure should be minimized on *well-shaped* tetrahedras and becomes really big on *bad-shaped* tetrahedras. Typically a *well-shaped* tetrahedron would be one close to be a regular tetrahedron, whereas the *bad-shaped* ones are typically slivers which are very flat tetrahedras that appears in constrained and unconstrained Delaunay tetrahedrisation (see on figure 2) or the ones with very acute angle.

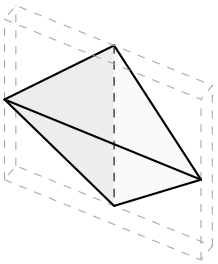


Figure 2: Example of sliver

In our implementation we choosed to use the Dirichlet energy because it's assigning 3 to a regular tetrahedron, is not upper bounded and assigns very high value to slivers.

Definition 2.9 (Edge matrix)**.** Let t be a tetrahedron composed of vertices $(\mathbf{x}_i)_{0 \leq i \leq 3} \in \mathbb{R}^3$, the edge matrix of t is defined to be:

$$T := \begin{pmatrix} \mathbf{x}_1 - \mathbf{x}_0 & \mathbf{x}_2 - \mathbf{x}_0 & \mathbf{x}_3 - \mathbf{x}_0 \end{pmatrix}$$

Remark 2.3. This definition of the edge matrix is invariant by translation.

Definition 2.10 (Tetrahedron deformation)**.** Let t be a tetrahedron and r another one that you want to deform t to, let T and respectively R be the associated edge matrix. We define the deformation matrix to be:

$$D := TR^{-1}$$

Definition 2.11 (Dirichlet energy)**.** Let t be a tetrahedron and J be the deformation matrix when we deform t to the regular tetrahedron of side 1. We define the Dirichlet energy to be:

$$\mathcal{E}(t) := \frac{\mathrm{Tr}(JJ^t)}{\det(J)^{\frac{2}{3}}}$$

Remark 2.4. The definition of this energy makes it so that it's invariant by rotation of t and of the regulart tetrahedron chosen, this is why we say *the* regular tetrahedron of side 1.

Remark 2.5. One may wonder if the order of the vertices in the input tetrahedron t is important, the answer is that as soon as the tetrahedron is well-oriented the order does not change the result. If the tetrahedron is badly-oriented the determinant at the denominator will be negative and so we just return the maximum value because it's out of definition for $\frac{2}{3}$ exponent.

3 The algorithm

3.1 General Principle

The algorithm consists of several local operations that I will detail in the following sections. The main function is the global optimization pass, which means it performs all the different local operations in sequence. One can choose to run this optimization pass a fixed number of times or to let it run until convergence (until no operations can be applied to the mesh).

The process for each individual operation pass is quite similar for all of them, as shown in Figure 3.

First, there is the simulation pass, which is crucial to determine which element on the mesh we will apply the operation to. Simulating the effect means computing the maximum quality measure locally before and after the hypothetical operation on a specific element. There are several steps to validate whether the element is suitable for the operation. Here they are:

- The maximum local quality measure must decrease.
- The geometry should not be broken (e.g., some tetrahedra may be inverted and thus have negative volume, or the boundary of the mesh may be deformed).
- The topology should be preserved (e.g., the manifold structure should be preserved).

The simulation must take into account all these conditions, and if an element meets every one of them, it is then added to a queue of elements to process. The queue is either a priority queue or a random queue (insert and random pop). For the priority queue, we tried two parameters for the priority: the maximum local quality measure before the operation, or the gap between the

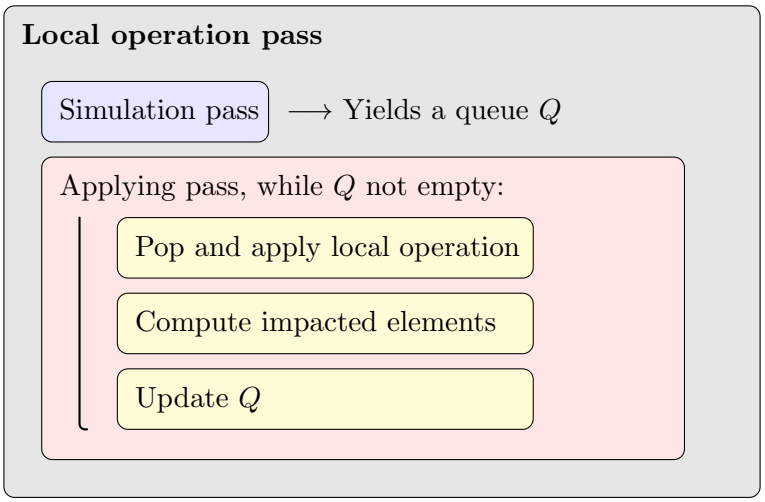


Figure 3: Local operation scheme

maximum local quality measure before and after the operation. In the end, we figured out by experiments that the second one is better in most cases.

After computing the queue, the process is greedy. We pop an element, and since it is in the queue, we know it is a good element to operate on. We apply the local operation and retrieve both the removed and impacted elements by the operation. These elements are then respectively removed from the queue and re-simulated to determine whether to add them back into the queue or remove them. We always want to maintain the invariant that if an element is in the queue, it is good to operate on.

The difficult parts are often knowing if the operation will break the topology and knowing which elements are impacted by a local operation.

In the following sections, I will present every operation, detailing the impacted elements, topological conditions to check, and what the operation does.

3.2 Local operations

3.2.1 Edge split

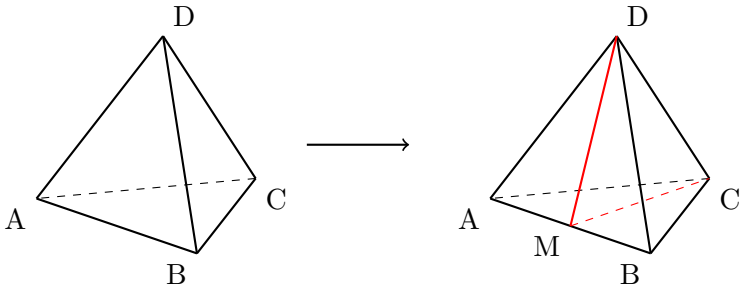


Figure 4: Example of splitting an edge

The edge split as the name suggests is the fact to add a new vertex to an edge and then arrange the geometry accordingly. For this operation we do not have to worry about the fact that it could break the topology because it cannot.

By defining $\mathcal{T} := \mathrm{ET}((u, v))$ the incident tetrahedra of the edge (u, v). We have that when we split (u, v) by adding w in the middle of it, each $T \in \mathcal{T}$ will be transformed in the following manner:

$$T := \{u, v, x, y\} \longrightarrow T' := \{u, w, x, y\} \text{ and } T'' := \{w, v, x, y\}$$

Let's define $\mathcal{T}' := \{T' \mid T \in \mathcal{T}\} \cup \{T'' \mid T \in \mathcal{T}\}$ the set of transformed tetrahedra. Therefore the operation is valid on the edge (u, v) if:

$$\max_{T' \in \mathcal{T}'} \mathcal{E}(T') \leq \max_{T \in \mathcal{T}} \mathcal{E}(T)$$

Then after applying the operation the edges to update are:

$$E := \mathrm{Edges}(\mathrm{ET}((u, w)) \cup \mathrm{ET}((w, v)))$$

Which are the edges of all the tetrahedra incident to either (u, w) or (w, v).

Here is an example of an edge split in the figure 4.

3.2.2 Edge collapse

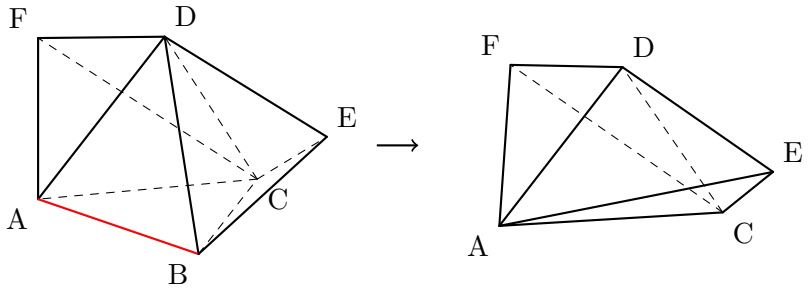


Figure 5: Example of collapsing an edge

Rather the intuitive approach to the edge collapse depicted on figure 5, this is one of the hardest operation to approach. In fact the difficulty of this operation is to prevent topological break. Firstly the vertex or the edge that we are going to remove should not be on the boundary of the model. Then the principal condition to ensure to not break the topology is the link condition for collapsing (u, v):

$$\mathrm{Lk}(\{(u, v)\}) = \mathrm{Lk}(\{u\}) \cap \mathrm{Lk}(\{v\}) \qquad \text{(Link condition)}$$

Then when collapsing the edge (u, v) on u the incident tetrahedra at the edge (u, v) $\mathrm{ET}((u, v))$ will be removed whereas each $T \in \mathrm{VT}(v) \backslash \mathrm{ET}((u, v))$ will be transformed in the following manner:

$$T := \{v, a, b, c\} \longrightarrow T' := \{u, a, b, c\}$$

Therefore the operation is valid if it respects the Link condition and if:

$$\max_{T \in \mathrm{VT}(v) \setminus \mathrm{ET}((u,v))} \mathcal{E}(T') \leq \max_{T \in \mathrm{VT}(v)} \mathcal{E}(T)$$

In a second time when collapsing the edge (u, v) on u the removed edges that has to be supressed from the queue are:

$$R := \mathrm{Edges}(\mathrm{St}(\{v\}))$$

and the edges to update are:

$$E := \mathrm{Edges}(\overline{\mathrm{Vert}(\{v\})}) \setminus R$$

3.2.3 Face swap

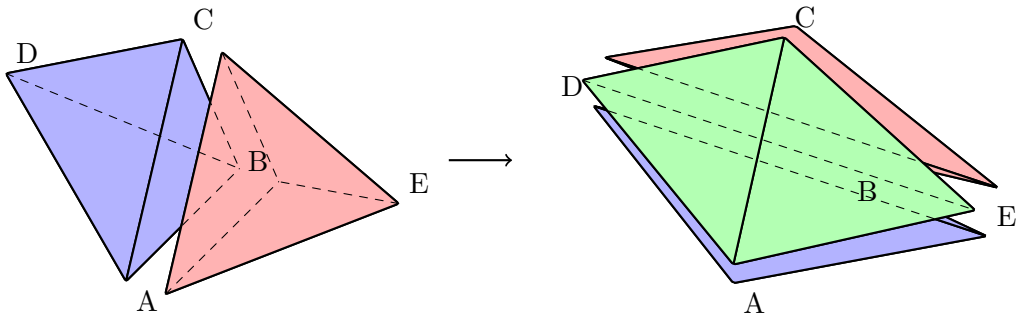


Figure 6: Example of swapping a face

The face swap modifies the configuration of two tetrahedra that share a common face $\sigma = (u, v, w)$.

The face swap is defined on two tetrahedra that share a face σ. Denote those tetrahedra as:

$$T_1 = \{u, v, w, a\} \text{ and } T_2 = \{u, v, w, b\}$$

where a and b are the vertices opposite to the shared face σ in each tetrahedron.

The face swap replaces T_1 and T_2 by three new tetrahedra:

$$T_1' = \{a, b, u, v\}, \quad T_2' = \{a, b, v, w\}, \quad T_3' = \{a, b, w, u\}$$

These tetrahedra share the edge (a, b) and together fill the same volume as $T_1 \cup T_2$, ensuring the local topology remains consistent. The operation is then feasible if:

$$\max_{T \in \{T_1', T_2', T_3'\}} \mathcal{E}(T) \leq \max_{T \in \{T_1, T_2\}} \mathcal{E}(T)$$

An example of such a face swap operation is illustrated in figure 6.

The faces to update are those of the three new tetrahedras added, which is:

$$F := \left\{ \sigma \in \overline{\{T_1', T_2', T_3'\}} \mid \dim \sigma = 2 \right\}$$

3.2.4 Edge swap

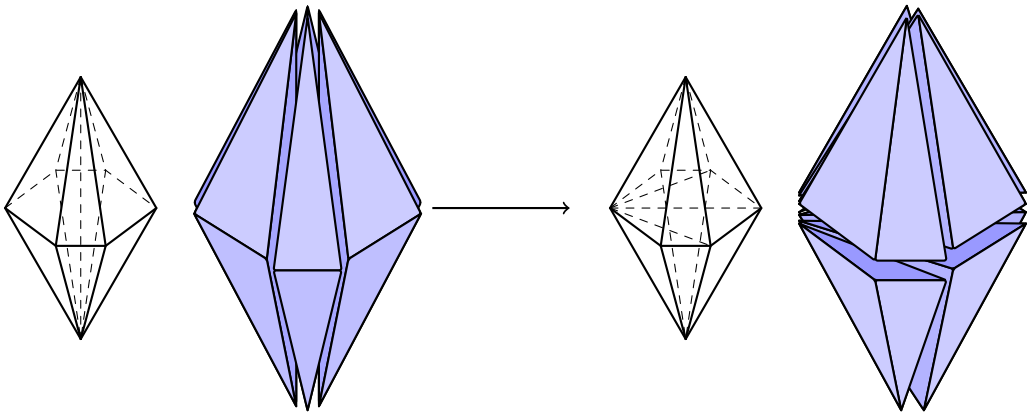


Figure 7: Example of swapping an edge

The edge swap changes the connectivity around an edge (u, v) shared by multiple tetrahedra. It consists of two steps:

1. Split the edge (u, v) by inserting a new vertex m at its midpoint
2. Collapse m onto a neighbor vertex w distinct from u and v

Let $\mathcal{T} := \mathrm{ET}((u, v))$, when collapsing on vertex w, each tetrahedron $T := \{u, v, a, b\}$ that does not contain w is subdivided into two tetrahedra:

$$T \to T' := \{u, w, a, b\} \quad \text{and} \quad T'' := \{v, w, a, b\}$$

Whereas the tetrahedron that contains the vertex w is removed. Let's define the updated tetrahedra when collapsing m on w:

$$\mathcal{T}_w := \left\{T' \mid T \in \mathcal{T},\ w \notin T\right\} \cup \left\{T'' \mid T \in \mathcal{T},\ w \notin T\right\}$$

Then concerning the topological point of view we still have to check the link condition when collapsing on m on w, we can then define the collapsible vertices to be:

$$\mathcal{C} := \{w \in \mathrm{Vert}(\mathrm{Lk}((u, v))) \mid \mathrm{Lk}(\{(m, w)\}) = \mathrm{Lk}(\{m\}) \cap \mathrm{Lk}(\{w\})\}$$

The operation is then valid if:

$$\min_{w \in \mathcal{C}} \max_{T \in \mathcal{T}_w} \mathcal{E}(T) \leq \max_{T \in \mathrm{ET}((u,v))} \mathcal{E}(T)$$

Finally there is no edge that is removed except the one we operated on and the edges to update are:

$$E := \bigcup_{x \in \mathrm{Vert}(\mathcal{T})} \{(x, y) \mid y \text{ is a neighbour of } x\}$$

There is an example in figure 7.

3.2.5 Vertex smoothing

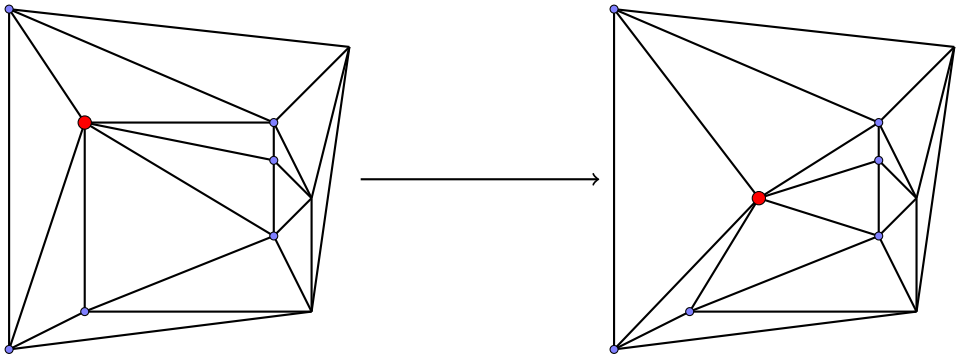


Figure 8: Example of smoothing a vertex

Vertex smoothing is an operation that does not change the connectivity of the mesh. Instead it changes the geometry, by moving vertices to the barycenter of their neighbour. Let v be a vertex and $V := \text{Vert}(\overline{v}) \setminus \{v\}$ the neighbour of v. Then v is moved to:

$$v' := \frac{1}{n} \sum_{u \in V} u$$

And so each $T := \{v, x, y, z\} \in \text{VT}(v)$ is transformed to $T' := \{v', x, y, z\}$ and the operation is valid if it does not invert any tetrahedron and if:

$$\max_{T \in \text{VT}(v)} \mathcal{E}(T') \leq \max_{T \in \text{VT}(v)} \mathcal{E}(T)$$

No vertices have been removed and thus the vertices to update are the ones in V.
Here is an example of smoothing a vertex in figure 8 (it is 2D here for simplicity).

3.2.6 Tetrahedron collapse

Slivers as depicted in figure 2 can be hard to get rid of. In fact in certain cases, due to the fact that we are only doing operations if they decrease the maximum quality measures locally, it is hard to remove slivers in one single operation. This is why creating a new combination of already existing operations can help to prevent slivers to appear in the final mesh.

A typical example of slivers that were hard to remove were the one near the boundary. Every single operation would cause to degrade the mesh locally. But seeing further than one operation is possible, and this is what the tetrahedron collapse does.

The tetrahedron collapse on $T := \{u, v, w, x\}$ is a series of three operation, which are:

1. An edge split on (u, v) by adding m
2. An edge split on (w, x) by adding m'
3. An edge collapse of m' on m

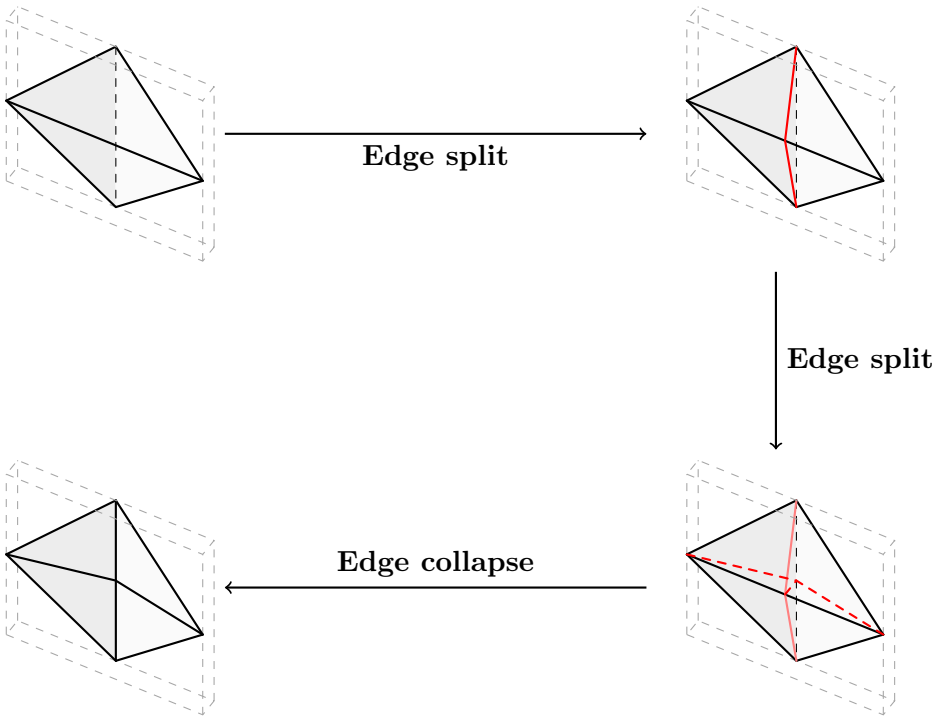


Figure 9: Tetrahedra collapsing process

By defining $\mathcal{R} := \text{ET}((u,v))\setminus\{T\}$ and $\mathcal{S} := \text{ET}((w,x))\setminus\{T\}$ and then define the updated tetrahedra by splitting and collapsing we can define $\mathcal{R}'$ and $\mathcal{S}'$:

$$\text{For } R := \{u,v,a,b\} \in \mathcal{R}, R \longrightarrow R' := \{u,m,a,b\} \in \mathcal{R}' \text{ and } R'' := \{v,m,a,b\} \in \mathcal{R}'$$

$$\text{For } S := \{w,x,a,b\} \in \mathcal{S}, S \longrightarrow S' := \{w,m,a,b\} \in \mathcal{S}' \text{ and } S'' := \{x,m,a,b\} \in \mathcal{S}'$$

Then the operation is valid if:

$$\max_{T\in\mathcal{R}'\cup\mathcal{S}'} \mathcal{E}(T) \leq \max_{T\in\mathcal{R}\cup\mathcal{S}\cup\{T\}} \mathcal{E}(T)$$

The link condition also needs to be checked when collapsing the edge (m,m').
Finally the set of tetrahedron to update is $\mathcal{R}' \cup \mathcal{S}'$.

4 Experiments

All the experiments I have conducted were on my machine, a M1 Mac with 8 GB of RAM. Here is a link to the GitHub repository where you will be able to reproduce the experiments and run the code.

4.1 Code organizations and specifications

The GitHub is a fork of the CDT repository of Marco Attene; this way, I can have access to the algorithm that produces constrained Delaunay tetrahedrization. The code itself is in the `code/src` directory. The code that I have written is in the following files:

- `optimize.cpp/h`
- `quality_measure.cpp/h`
- `random_map.h`
- `updatable_queue_template.h`
- `updatable_priority_queue.h`

Concerning the greedy method to use, we have tried several ones. The first thing we tried was to use a vector instead of a queue, to sort it according to the priority parameter I described at the end of § 3.1, and then not update it and check when popping an element if the element is still worth operating on. This method was converging very slowly, so we decided to switch to an updatable priority queue.

The priority queue needs to have an ID for each element; then it can attach some data to this ID (e.g., position of the split vertex for an edge...). The insertion and the removal of an element are done in $\mathcal{O}(\log n)$, where n is the number of elements in the queue, and update is done in constant time. After some tests, it seems that sometimes, as the priority queue is fully deterministic, it can be trapped in a loop that updates the elements in a certain order, which prevents the execution of other operations that would have been possible previously. To solve this problem, we thought about a random map/queue that would be updatable and have random pop.

We then coded a simple structure of random queue, which is composed of a vector containing the elements and an unordered map that maps the position of each element in the vector. In this way, insertion, removal, and update are all done in constant amortized time.

As it is a greedy process, there is no clear winner—it depends on the model you are processing.

4.2 One example

I will first show some results on one example, which is plotted in Figure 10. This is quite a complex model with 4557 vertices and 9110 triangles. The CDT process has added 36722 Steiner points, and we are starting from a point where the maximum quality measure is 3.81836e+06 and the average quality measure is 1599.68. After 49 minutes and 43 seconds, our algorithm has converged to a maximum quality measure of 4814.68 and an average of 10.6005. This is respectively a decrease of 79300% and 15000%.

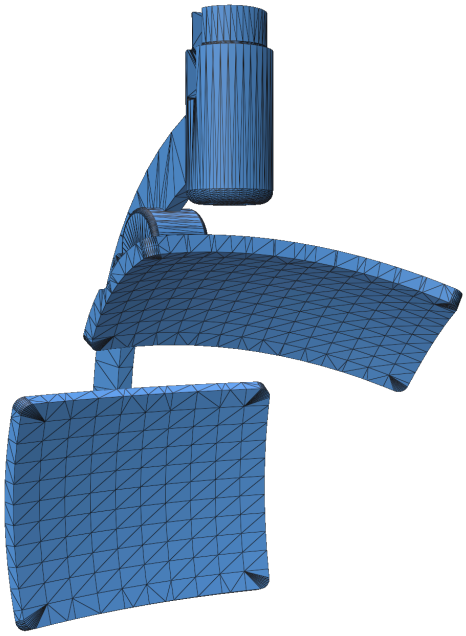


Figure 10: The model 112544

Here, in Figure 11, is the average energy evolution:

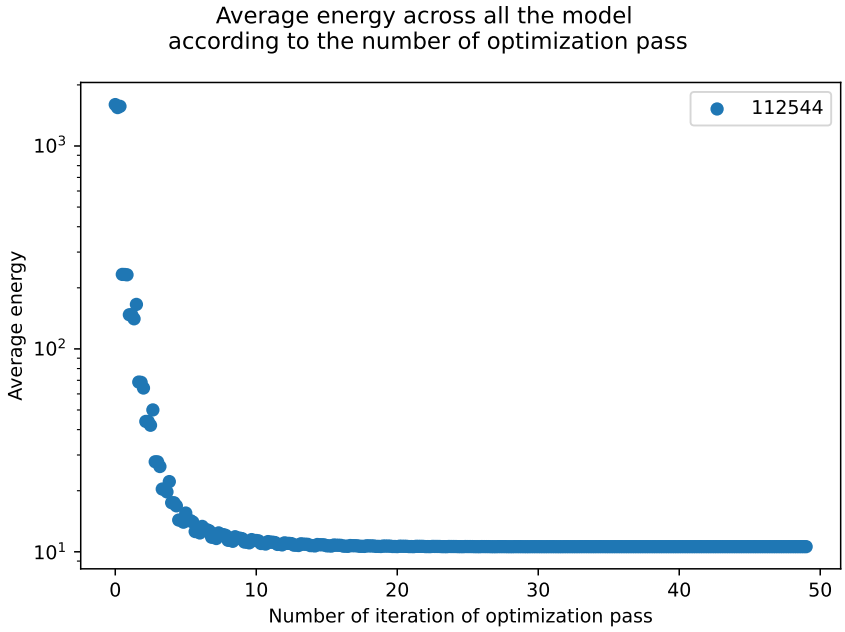


Figure 11: Average energy evolution on model 112544

4.3 Comparison with *TetGen*

When it comes to comparing with existing software, I figured out that I would test both programs on the same models. More precisely, I decided to perform tests on the 500 smallest models of the Thingi10k dataset. Then I will measure three components: the duration of the process, the average quality measure across all the models at the end, and also the maximum. I will also take into account failures from both algorithms.

	CDT	TetGen
Fails	0%	25,2%

Table 1: Stats on the smallest 500 models of *Thingi10k*

	CDT	TetGen
Best on time	14,44%	85,56%
Best on average	5,35%	94,65%
Best on max	20,59%	79,41%

Table 2: Stats on the remaining models TetGen did not failed on

Results show a clear trade-off between robustness and quality. Some explanations can be found; the first thing is that TetGen by default looks for operations 3 steps ahead, whereas our implementation only looks one step ahead for most of the operations (only the tetrahedron collapse looks 3 steps ahead). Concerning the duration, I did not have much time in 3 months to produce a robust algorithm while optimizing it. I preferred firstly to have a working version.

5 Conclusion

To conclude, I think that a good part of the objective is complete. We managed to maintain the robustness of the process in the refinement part while improving the quality of the mesh. We also kept the boundaries of the input untouched as it was stated initially. To accomplish this, I needed to implement simulators for the operations to predict their behavior on the geometry and the topology of the mesh. After that, I needed to actually implement the local operation itself. Still, some things may have to be done, like optimizing the speed or continuing to improve the quality of the mesh. I detail in the following paragraph some ideas we did not have the time to develop that would be interesting to look at in the future.

Possible improvements

- **Refining some operations**: Some operation designs have not been developed to their full potential. I particularly think of the edge split and the vertex smoothing. For the moment, the split vertex is always placed at the middle of the edge, and the vertex smoothing also always moves the vertex to the barycenter of its neighbors. Another finer option would be to run an algorithm like Newton's method to find the best position for the concerned vertex.
- **Be less strict on the updates**: Currently, the major bottleneck concerning the speed is the number of updates we have to make. In fact, for some operations, the number of calls to the

simulation function for the update after operating on one single element can reach thousands of calls. The thing is that, in general, we need to do all those updates because all elements might really be impacted. In reality, it seems that only a very few of those updates are really necessary. These are often the elements of the tetrahedra that are really impacted and not, for example, edges that are connected to one vertex of an impacted tetrahedron (which is the case for every operation using the collapse method). The idea may be to stop updating the queue and, when popping, check that the operation is still good to perform. In this way, we may have more global optimization passes, but they would be much quicker. Concerning the efficiency of this method, we do not think it would affect much because, in the end, it is still a greedy process.

I would also like to thank all the team, Marco and Cino for making this experience joyful and peaceful. It would be a pleasure to come back in Genova which is a city I really appreciated!

References

[1] Lorenzo Diazzi et al. *Constrained Delaunay Tetrahedrization: A Robust and Practical Approach*. 2023. arXiv: 2309.09805 [cs.CG]. URL: https://arxiv.org/abs/2309.09805.

[2] Yixin Hu et al. "Tetrahedral meshing in the wild". In: *ACM Trans. Graph.* 37.4 (July 2018). ISSN: 0730-0301. DOI: 10.1145/3197517.3201353. URL: https://doi.org/10.1145/3197517.3201353.

[3] The CGAL Project. *CGAL User and Reference Manual*. 6.0.1. CGAL Editorial Board, 2024. URL: https://doc.cgal.org/6.0.1/Manual/packages.html.

[4] Hang Si. "TetGen, a Delaunay-Based Quality Tetrahedral Mesh Generator". In: *ACM Trans. Math. Softw.* 41.2 (Feb. 2015). ISSN: 0098-3500. DOI: 10.1145/2629697. URL: https://doi.org/10.1145/2629697.